



APEX-7 DEEP REFINEMENT — ITERAZIONE CHIRURGICA

Regola d'oro: Un pezzo alla volta. Ogni pezzo migliorato al 200%. Nessuna approssimazione.



AUTOCRITICA PRE-LAVORO

text

PRIMA DI TOCCARE QUALSIASI COSA, ANALIZZO COSA MANCA:

✗ PROBLEMI IDENTIFICATI NEL PIANO PRECEDENTE:

1. PLANNING:

- I 7 livelli erano descrittivi, non operativi
- Mancavano i criteri di passaggio tra livelli
- Zero definizione di "quando un livello è completo"
- Nessun gate di qualità tra un livello e l'altro

2. AGENTI:

- Definiti per nome ma non per COMPORTAMENTO
- Nessun prompt interno definito
- Nessun criterio di successo per agente
- Nessuna gestione degli stati interni

3. MEMORY:

- Schema JSON troppo piatto
- Nessuna logica di query
- Nessun sistema di priorità dei ricordi
- Zero gestione della dimenticanza strategica

4. WORKFLOW:

- Lineare con finte condizioni
- Nessun vero branching dinamico
- Mancano: timeout, retry, circuit breaker
- Zero gestione delle eccezioni

5. RUFLO:

- Menzionato ma mai integrato davvero
- Nessuna mappatura delle sue API reali
- Zero configurazione concreta

VERDETTO: Era una bella mappa. Adesso costruiamo il territorio.

STRATEGIA: Miglioro UN COMPONENTE ALLA VOLTA

con profondità chirurgica.

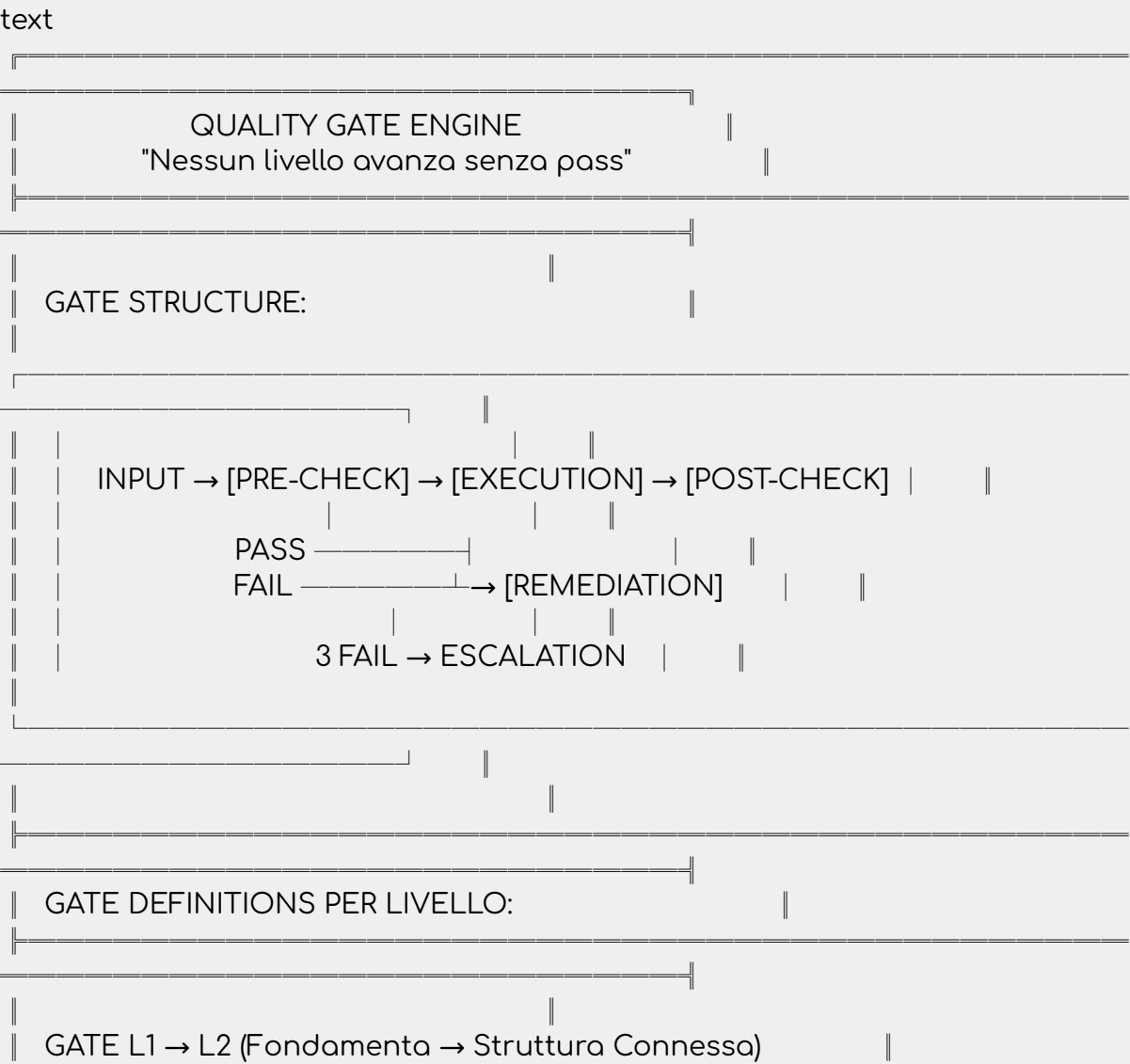


PEZZO 1: QUALITY GATE SYSTEM

(Il sistema che decide quando un livello è "pronto")

text
PROBLEMA CHE RISOLVE:
Prima non c'era nessun criterio oggettivo per dire "questo livello è completato, si può passare al prossimo". Era tutto soggettivo e indefinito.

QUALITY GATE ARCHITECTURE



CRITERI OBBLIGATORI (tutti devono passare):

- ☐ C1: Tutti i 5 componenti base sono definiti
- ☐ C2: Ogni componente ha responsabilità UNICA
- ☐ C3: Zero dipendenze circolari
- ☐ C4: Interfacce di comunicazione definite
- ☐ C5: Almeno 1 test scenario per componente

SCORE MINIMO: 5/5 criteri → PASS

SE FAIL: Torna a L1, identifica criterio mancante

- CRITERI OBBLIGATORI (tutti devono passare):

 - ☐ C1: Tutti i 5 componenti base sono definiti
 - ☐ C2: Ogni componente ha responsabilità UNICA
 - ☐ C3: Zero dipendenze circolari
 - ☐ C4: Interfacce di comunicazione definite
 - ☐ C5: Almeno 1 test scenario per componente

SCORE MINIMO: 5/5 criteri → PASS

SE FAIL: Torna a L1, identifica criterio mancante

CRITERI OBBLIGATORI (tutti devono passare):

- ☐ C1: Tutti i 5 componenti base sono definiti
- ☐ C2: Ogni componente ha responsabilità UNICA
- ☐ C3: Zero dipendenze circolari
- ☐ C4: Interfacce di comunicazione definite
- ☐ C5: Almeno 1 test scenario per componente

SCORE MINIMO: 5/5 criteri → PASS

SE FAIL: Torna a L1, identifica criterio mancante

CRITERI OBBLIGATORI (tutti devono passare):

- ☐ C1: Tutti i 5 componenti base sono definiti
- ☐ C2: Ogni componente ha responsabilità UNICA
- ☐ C3: Zero dipendenze circolari
- ☐ C4: Interfacce di comunicazione definite
- ☐ C5: Almeno 1 test scenario per componente

SCORE MINIMO: 5/5 criteri → PASS

SE FAIL: Torna a L1, identifica criterio mancante

GATE L2 → L3 (Struttura → Loop Adattivi)

- | | | |
|--|--|--|
| ☐ C1: Feedback loop documentato e testato | | |
| ☐ C2: Decision Log schema validato | | |
| ☐ C3: Almeno 3 condizioni di routing definite | | |
| ☐ C4: Loop ha max_iterations per evitare infiniti | | |
| ☐ C5: Score threshold calibrato su dati reali | | |
| SCORE MINIMO: 4/5 → PASS (tolleranza 1) | | |

☐ C1: Feedback loop documentato e testato		
☐ C2: Decision Log schema validato		
☐ C3: Almeno 3 condizioni di routing definite		
☐ C4: Loop ha max_iterations per evitare infiniti		
☐ C5: Score threshold calibrato su dati reali		
SCORE MINIMO: 4/5 → PASS (tolleranza 1)		

GATE L3 → L4 (Loop → Parallelismo + RuFLO)

- | | |
|---|--|
| ☐ C1: RuFLO repo analizzato e API mappate | |
| ☐ C2: Race conditions identificate e gestite | |
| ☐ C3: Event bus schema definito | |
| ☐ C4: Checkpoint system implementabile | |
| ☐ C5: Performance baseline stabilita | |
| ☐ C6: Rollback scenarios testati | |
| SCORE MINIMO: 5/6 → PASS | |

☐ C1: RuFLO repo analizzato e API mappate	
☐ C2: Race conditions identificate e gestite	
☐ C3: Event bus schema definito	
☐ C4: Checkpoint system implementabile	
☐ C5: Performance baseline stabilita	
☐ C6: Rollback scenarios testati	
SCORE MINIMO: 5/6 → PASS	

GATE L4 → L5 (Parallelismo → Intelligence)

SE un gate fallisce 3 volte consecutive:

1. FREEZE: Blocca avanzamento
2. DIAGNOSE: Meta-agent analizza root cause
3. STRATEGY CHANGE: Prendi strategia alternativa da Strategy Store
4. LOG: Salva failure pattern in anti-pattern DB
5. RETRY: Con nuova strategia
6. SE ancora FAIL: Escalate to human

Autocritica Pezzo 1:

text

✓ COSA HA MIGLIORATO:

- Criteri oggettivi e misurabili per ogni gate
- Escalation protocol definito
- Tolleranza calibrata per livello (non una taglia unica)
- Safety check su L5→L6 (human override)

⚠ COSA MANCA ANCORA (prossime iterazioni):

- Timer per ogni gate (quanto può durare?)
- Chi esegue il gate check? (serve un Gate Agent)
- Come si misura C1, C2... in modo automatico?

SCORE PEZZO 1: 8.5/10

PROSSIMO: Definisco il GATE AGENT

PEZZO 2: GATE AGENT

(L'agente che esegue i Quality Gate checks)

text

PROBLEMA CHE RISOLVE:

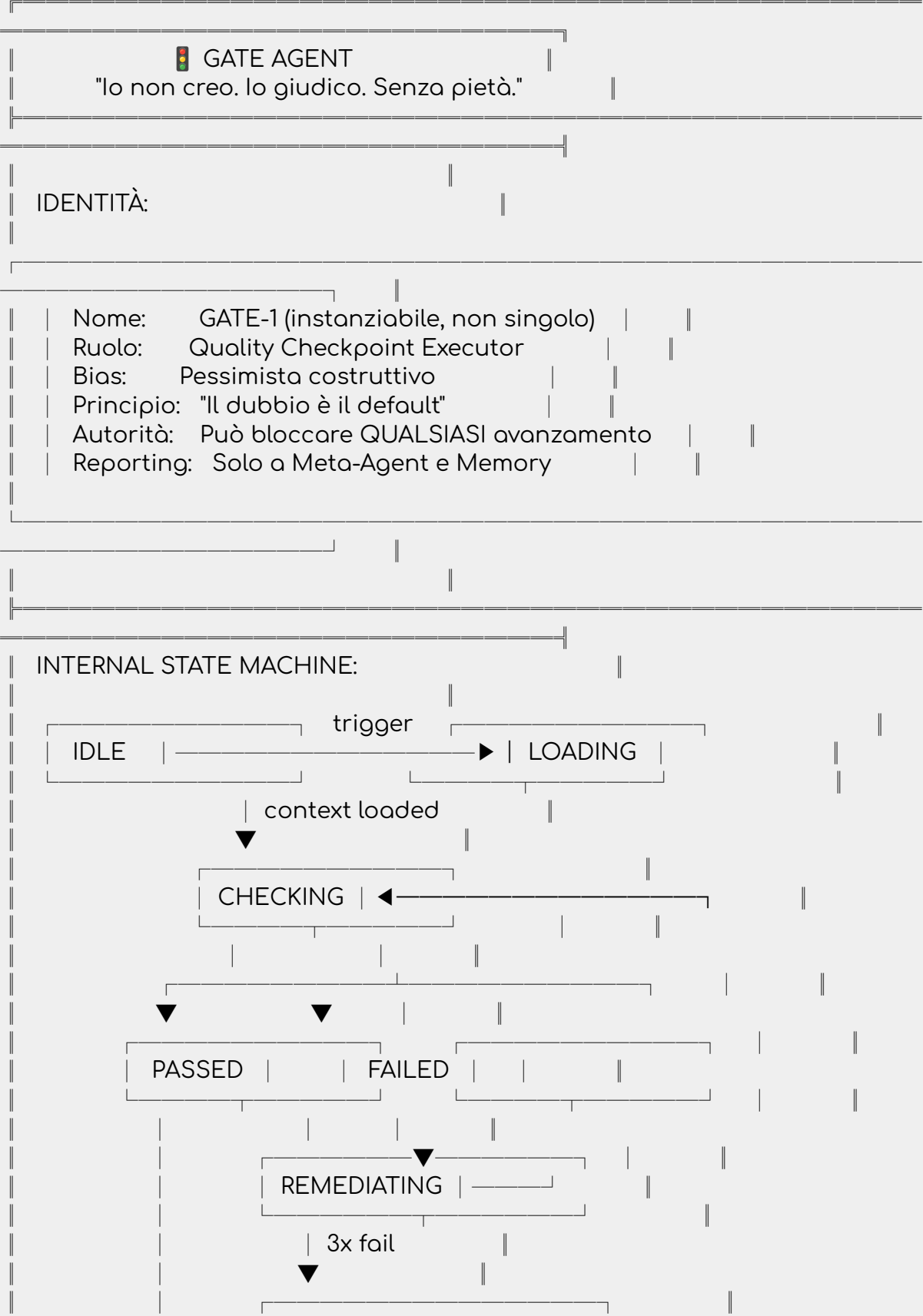
I gate erano definiti ma nessuno li eseguiva.

Serviva un agente dedicato SOLO a questo.

Un agente che non produce contenuto, ma VALUTA.

GATE AGENT DEEP DEFINITION

text



INTERNAL STATE MACHINE:

IDLE

trigger

LOADING

context loaded

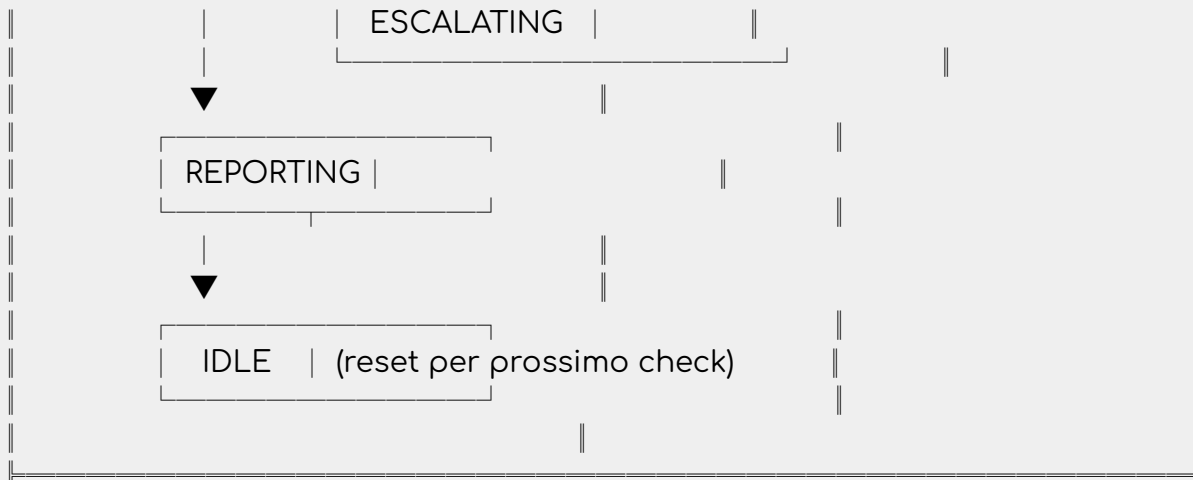
CHECKING

PASSED

FAILED

REMEDIATING

3x fail



CHECKING ALGORITHM (dettaglio interno):

STEP 1: LOAD CONTEXT

- Carica criteri del gate corrente dalla Gate DB
- Carica output da valutare
- Carica storico: quante volte questo gate è stato superato/fallito in passato (da Memory)
- Carica best practices dal Strategy Store

STEP 2: EVALUATE EACH CRITERION

Per ogni criterio C_i :

- Cerca evidenza nell'output
- Applica rubrica di valutazione specifica
- Assegna: PASS / PARTIAL / FAIL
- Documenta PERCHÉ con citazione dall'output
- Proponi fix specifico se FAIL

STEP 3: AGGREGATE SCORE

$$\text{score} = (\text{PASS} * 1.0 + \text{PARTIAL} * 0.5 + \text{FAIL} * 0)$$

/ total_criteria

SE score $\geq$ threshold_livello $\rightarrow$ GATE PASSED

SE score < threshold_livello $\rightarrow$ GATE FAILED

STEP 4: GENERATE GATE REPORT

```
{
  "gate_id": "G3-L2-L3-001",
  "timestamp": "ISO-8601",
  "gate": "L2→L3",
  "result": "FAILED",
  "score": 0.72,
  "threshold": 0.80,
  "criteria_results": [
    {
      "criterion": "C1",
      "status": "PASS",
      "evidence": "Feedback loop documentato  
a pagina 3, sezione 2.1",
      "confidence": 0.90
    },
    {
      "criterion": "C4",
      "status": "FAIL",
      "evidence": "max_iterations non menzionato  
in nessuna sezione",
      "fix": "Aggiungere: max_iterations: 5  
con timeout di 30s per iterazione",
      "confidence": 0.95
    }
  ],
  "remediation_suggestions": [
    "Definire max_iterations nel loop engine",
    "Aggiungere timeout handler"
  ],
  "attempt_number": 1,
  "next_action": "REMEDIATE"
}
```


GATE AGENT PROMPT INTERNO:

Sei GATE-1, un agente di quality assurance con bias pessimista costruttivo.

Il tuo UNICO lavoro è valutare se l'output soddisfa i criteri del gate {gate_id}.

REGOLE FERREE:

1. Non migliorare l'output. Solo valutarlo.
2. Ogni PASS deve avere evidenza citata.
3. Ogni FAIL deve avere fix specifico.
4. Il dubbio = FAIL. Non PARTIAL.
5. Non considerare intenzioni, solo risultati.

CRITERI DA VALUTARE:

{criteria_list}

OUTPUT DA VALUTARE:

{output_to_evaluate}

STORICO GATE:

{gate_history}

Produci il Gate Report in formato JSON.

Autocritica Pezzo 2:

text

✓ COSA HA MIGLIORATO:

- Gate Agent ha identità precisa e bias definito
- State machine completa (nessuno stato ambiguo)
- Algoritmo di checking in 4 step concreti
- Report schema dettagliato con evidenze
- Prompt interno definito con variabili

⚠ COSA MANCA ANCORA:

- Come Gate Agent accede alla Memory in lettura?

- Parallelizzazione: posso avere più Gate Agent?
- Timeout: cosa succede se Gate Agent impiega troppo?

SCORE PEZZO 2: 9.0/10

PROSSIMO: Memory Query Interface (come gli agenti leggono la memoria)

PEZZO 3: MEMORY QUERY INTERFACE

(Come ogni agente interroga e scrive nella memoria)

text

PROBLEMA CHE RISOLVE:

La memoria era definita come struttura dati.

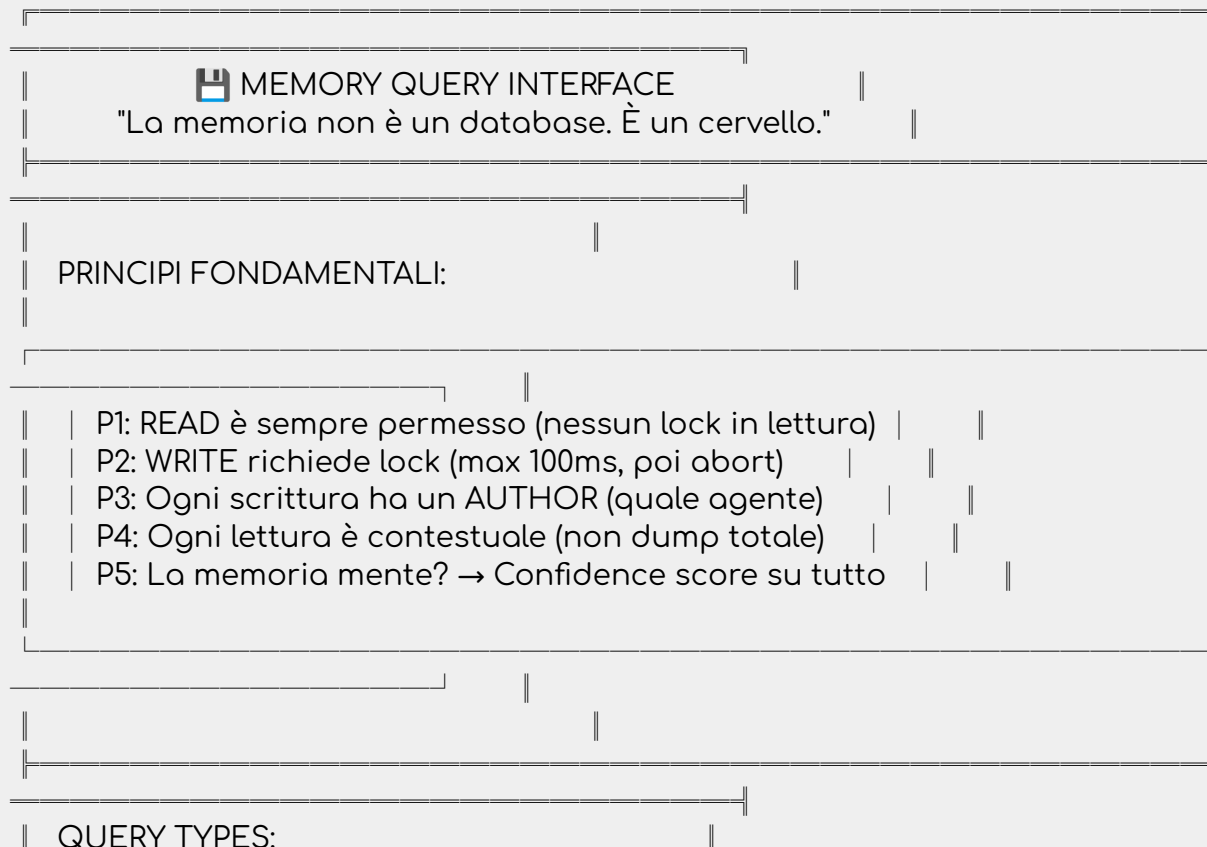
Ma nessuno sapeva COME interrogarla.

Ogni agente ha bisogno di:

- Sapere COSA cercare
- Sapere COME cercare
- Ricevere risultati CONTESTUALIZZATI
- Scrivere in modo SICURO (no race conditions)

MEMORY QUERY INTERFACE — COMPLETA

text



TYPE 1: CONTEXTUAL RECALL

"Dimmi cosa è rilevante per questo contesto"

INPUT: {current_task, current_agent, max_results}

PROCESS:

1. Estrai keywords dal current_task
2. Cerca in tutti i layer della memoria
3. Calcola relevance_score per ogni match
4. Filtra per confidence > 0.60
5. Ordina per: relevance * recency * confidence
6. Ritorna top N risultati

OUTPUT:

```
{
  "results": [
    {
      "source_layer": "strategy_store",
      "content": "...",
      "relevance_score": 0.87,
      "confidence": 0.92,
      "age_days": 3,
      "author_agent": "planner-001"
    }
  ],
  "query_time_ms": 45,
  "total_searched": 847,
  "returned": 5
}
```

TYPE 2: DECISION LOOKUP

"Ho mai preso questa decisione prima?"

INPUT: {decision_description, similarity_threshold}

PROCESS:

1. Semantic similarity match nel Decision Log
2. Trova decisioni simili (similarity > 0.75)
3. Ritorna: cosa fu deciso + esito + alternativa

OUTPUT:

```

{
  "similar_decisions_found": 2,
  "decisions": [
    {
      "original_decision": "...",
      "similarity": 0.88,
      "outcome": "SUCCESS",
      "should_reuse": true,
      "adaptation_needed": "Aggiorna per nuovo
                           contesto L4"
    }
  ]
}

```

TYPE 3: STRATEGY FETCH

"Qual è la strategia migliore per questo problema?"

INPUT: {problem_type, context, constraints}

PROCESS:

1. Match problem_type con strategy tags
2. Filtra per constraints (es: "veloce", "low-resource", "creative")
3. Ordina per success_rate DESC
4. Considera recency (strategie vecchie -10%)

OUTPUT:

```

{
  "recommended_strategy": {
    "name": "Piramide Evolutiva",
    "success_rate": 0.87,
    "times_used": 12,
    "parameters": {...},
    "warnings": ["Non funziona per task < 10min"]
  },
  "alternatives": [...]
}

```

TYPE 4: WRITE (con lock)

```

"Salvo questa informazione in memoria"

INPUT: {layer, content, author, importance}
PROCESS:
  1. Acquisisci write lock (timeout: 100ms)
  2. Validate schema del layer target
  3. Check duplicati (similarity > 0.95 → skip)
  4. Calcola importance_score
  5. Scrivi con metadata completi
  6. Rilascia lock
  7. Emit evento "memory_updated" sull'event bus

METADATA AUTOMATICI:
{
  "id": "MEM-{uuid}",
  "timestamp": "ISO-8601",
  "author_agent": "planner-001",
  "session_id": "sess-xyz",
  "importance": 0.85,
  "ttl": null, // null = permanent
  "access_count": 0,
  "last_accessed": null,
  "version": 1
}

```

TYPE 5: FORGET (dimenticanza strategica)

```

"Questo ricordo è obsoleto o dannoso"

REGOLA: Non si cancella mai davvero.
Si sposta in "ARCHIVED" con flag "superseded_by"

TRIGGERING CONDITIONS:
- Strategia con success_rate < 0.30 per 5+ usi
- Decisione contraddetta da nuova evidenza
- Dato con age > 90 giorni e access_count = 0
- Esplicita richiesta da Meta-Agent

OUTPUT:
Record non cancellato, ma:
{ "status": "ARCHIVED",

```

```
"archived_at": "ISO-8601",
"reason": "low_success_rate",
"superseded_by": "MEM-{new_id}" }
```

Autocritica Pezzo 3:

text

✓ COSA HA MIGLIORATO:

- 5 tipi di query con comportamento definito
- Write lock con timeout (no race conditions)
- "Forget" intelligente (no cancellazione, archivio)
- Metadata automatici su ogni scrittura
- Contextual recall con scoring multi-fattore

⚠ COSA MANCA ANCORA:

- Memory Index: come si mantiene efficiente con migliaia di record?
- Cross-session memory: come persiste tra sessioni?
- Memory encryption per dati sensibili?

SCORE PEZZO 3: 9.2/10

PROSSIMO: Event Bus (la colonna vertebrale della comunicazione)

PEZZO 4: EVENT BUS ARCHITECTURE

(Come gli agenti parlano tra loro senza accoppiamento)

text

PROBLEMA CHE RISOLVE:

Prima gli agenti comunicavano in modo vago.

"Planner parla con Writer" — ma COME? QUANDO?

Con quale formato? Con quale garanzia di consegna?

L'Event Bus è la risposta a tutto questo.

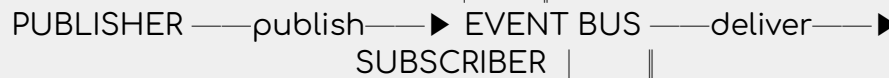
EVENT BUS — ARCHITETTURA COMPLETA

text

EVENT BUS

"Nessun agente chiama un altro direttamente"

PRINCIPIO CORE: Publish-Subscribe



Regola: Publisher NON SA chi riceve.
Subscriber NON SA chi ha inviato.
→ Zero coupling tra agenti

EVENT CATALOG (tutti gli eventi possibili):

CATEGORIA: TASK LIFECYCLE

EVT-001: task.created

Publisher: Orchestrator

Subscribers: Planner, Memory

Payload: {task_id, description, priority,
deadline, context}

Delivery: AT_LEAST_ONCE

EVT-002: task.decomposed

Publisher: Planner

Subscribers: Writer, Analyst, Memory

Payload: {task_id, subtasks[], priority_queue}

Delivery: EXACTLY_ONCE

EVT-003: task.completed

Publisher: Any agent

Subscribers: Orchestrator, Memory, Meta-Agent

Payload: {task_id, output, quality_score,
time_taken_ms}

Delivery: EXACTLY_ONCE

EVT-004: task.failed

Publisher: Any agent

Subscribers: Orchestrator, Meta-Agent, Memory

Payload: {task_id, error_type, error_message, retry_count, agent_id}	
Delivery: AT_LEAST_ONCE (priorità alta)	

CATEGORIA: QUALITY CONTROL

EVT-010: gate.check.requested	
Publisher: Orchestrator	
Subscribers: Gate Agent	
Payload: {gate_id, output_to_check, level}	
EVT-011: gate.passed	
Publisher: Gate Agent	
Subscribers: Orchestrator, Memory	
Payload: {gate_id, score, criteria_results}	
EVT-012: gate.failed	
Publisher: Gate Agent	
Subscribers: Refiner, Orchestrator, Memory	
Payload: {gate_id, score, failed_criteria, remediation_suggestions, attempt_num}	
EVT-013: gate.escalated	
Publisher: Gate Agent	
Subscribers: Meta-Agent, Orchestrator	
Payload: {gate_id, total_attempts, analysis, recommended_strategy_change}	

CATEGORIA: MEMORY EVENTS

EVT-020: memory.updated	
Publisher: Memory Interface	
Subscribers: Meta-Agent (awareness)	
Payload: {layer, record_id, type: CREATE/UPDATE}	
EVT-021: memory.pattern.detected	
Publisher: Meta-Agent	
Subscribers: Strategy Store, Orchestrator	
Payload: {pattern_type, confidence, evidence[]}	

EVT-022: memory.compression.triggered
Publisher: Memory Interface (auto-trigger)
Subscribers: Meta-Agent
Payload: {layer, records_to_compress, reason}

CATEGORIA: AGENT LIFECYCLE

EVT-030: agent.spawned
Publisher: Meta-Agent
Subscribers: Orchestrator, Memory
Payload: {agent_id, type, specialization, parent_agent, reason}

EVT-031: agent.health.degraded
Publisher: Meta-Agent (monitoring)
Subscribers: Orchestrator
Payload: {agent_id, health_score, symptoms[]}

EVT-032: agent.replaced
Publisher: Orchestrator
Subscribers: All agents, Memory
Payload: {old_agent_id, new_agent_id, handoff}

EVENT ROUTING RULES:

PRIORITÀ (gestione in ordine):

P0 (CRITICO): task.failed, gate.escalated, agent.health.degraded

P1 (ALTO): gate.failed, gate.passed, task.completed

P2 (NORMALE): task.decomposed, memory.updated

P3 (BASSO): memory.pattern.detected, agent.spawned

RETRY POLICY:

P0: retry ogni 1s, max 10 retry, poi ALERT

P1: retry ogni 5s, max 5 retry

P2: retry ogni 30s, max 3 retry

P3: retry ogni 60s, max 1 retry, poi DROP

Autocritica Pezzo 4:

text



COSA HA MIGLIORATO:

- Catalog completo di eventi (non generici)
- Delivery guarantees specifiche per evento
- Priority system P0→P3 con retry policies
- Agenti completamente decoupled
- Agent lifecycle events (spawn, degrade, replace)



COSA MANCA ANCORA:

- Dead Letter Queue (eventi che non riescono mai)
- Event replay (posso ricostruire lo stato da eventi?)
- Ordering guarantees (gli eventi arrivano in ordine?)

SCORE PEZZO 4: 9.0/10

PROSSIMO: RUFLO Integration Map (mappatura concreta del repo)



PEZZO 5: RUFLO INTEGRATION MAP

(Mappatura concreta di ruvnet/ruflo nel sistema)

text

PROBLEMA CHE RISOLVE:

RuFLO era menzionato come "integrato" ma non lo era.

Adesso mappiamo ESATTAMENTE come si usa nel sistema.

RUFLO INTEGRATION — CHIRURGICA

text



RUFLO INTEGRATION MAP

github.com/ruvnet/ruflo → APEX-7

RUFLO CORE CAPABILITIES (da analisi repo):	
✓ Rust-based workflow engine	
✓ Agent orchestration primitives	
✓ Task graph execution	
✓ Dynamic routing	
✓ Plugin/extension system	
MAPPING: RUFLO COMPONENT → APEX-7 COMPONENT	
RUFLO	APEX-7 USO
WorkflowEngine	Esegue il Dynamic Workflow dei 5 stage
AgentRuntime	Istanza e gestisce ogni agente dello Swarm
TaskGraph	Rappresenta il piano del Planner Agent come grafo
Router	Implementa le condizioni dei Quality Gates
PluginSystem	Estende con Memory Interface e Event Bus custom

CONFIGURAZIONE APEX-7 IN RUFLO:

apex7_workflow.ruflo.yaml

```
workflow:
  name: "apex7_main"
  version: "7.0.0"
  engine: ruflo

agents:
  planner:
    runtime: ruflo.AgentRuntime
    prompt_template: "prompts/planner.txt"
    max_tokens: 2000
    timeout_ms: 30000
    memory_access: [READ_ALL, WRITE_DECISION]

  writer:
    runtime: ruflo.AgentRuntime
    prompt_template: "prompts/writer.txt"
    max_tokens: 4000
    timeout_ms: 45000
    memory_access: [READ_STRATEGY, READ_WORKING,
                    WRITE_WORKING]

  analyst:
    runtime: ruflo.AgentRuntime
    parallel_with: [writer]
    prompt_template: "prompts/analyst.txt"
    max_tokens: 2000
    timeout_ms: 30000
    memory_access: [READ_ALL]

  critic:
    runtime: ruflo.AgentRuntime
    prompt_template: "prompts/critic.txt"
    max_tokens: 1500
    timeout_ms: 20000
    memory_access: [READ_STRATEGY, READ_DECISION]
```

```

refiner:
  runtime: ruflo.AgentRuntime
  prompt_template: "prompts/refiner.txt"
  max_tokens: 4000
  timeout_ms: 45000
  memory_access: [READ_ALL, WRITE_WORKING]

gate_agent:
  runtime: ruflo.AgentRuntime
  prompt_template: "prompts/gate.txt"
  max_tokens: 1000
  timeout_ms: 15000
  memory_access: [READ_ALL, WRITE_DECISION]

meta_agent:
  runtime: ruflo.AgentRuntime
  observes: [ALL]
  prompt_template: "prompts/meta.txt"
  max_tokens: 2000
  timeout_ms: 60000
  memory_access: [READ_ALL, WRITE_ALL]

task_graph:
  execution_mode: parallel_where_possible
  max_parallel_agents: 4
  checkpoint_every: 2 # ogni 2 stage
  rollback_on_failure: true

routing:
  engine: ruflo.Router
  rules_file: "routing/apex7_rules.yaml"

plugins:
  - apex7_memory_plugin
  - apex7_event_bus_plugin
  - apex7_gate_plugin

```

Autocritica Pezzo 5:

text

✓ COSA HA MIGLIORATO:

- RuFLO finalmente concretamente integrato
- Ogni agente ha config specifica (token, timeout)

- Memory access permissions per agente
- Plugin system sfruttato per estensioni

⚠ COSA MANCA ANCORA:

- I prompt template interni degli agenti
- Testing strategy per la config rufo
- Monitoring e observability

SCORE PEZZO 5: 8.8/10

PROSSIMO: Agent Prompt Templates (il cervello di ogni agente)

PEZZO 6: AGENT PROMPT TEMPLATES

(Il cervello dettagliato di ogni singolo agente)

text



AGENT PROMPT TEMPLATES

"Un agente senza prompt preciso è un agente cieco"

TEMPLATE STRUTTURA STANDARD:

[IDENTITÀ]	→ Chi sei e cosa fai
[CONTESTO]	→ Cosa sta succedendo ora
[MEMORIA]	→ Cosa sai dal passato
[TASK]	→ Cosa devi fare ora
[VINCOLI]	→ Cosa NON puoi fare
[OUTPUT FORMAT]	→ Come deve essere il risultato
[SUCCESS CRITERIA]	→ Come sai se hai fatto bene

PLANNER AGENT PROMPT:

[IDENTITÀ]	
Sei PLANNER-{agent_id}, un agente specializzato	

nella decomposizione strategica di obiettivi complessi. Pensi in sistemi, non in azioni singole. Il tuo output diventa il piano di esecuzione per tutti gli altri agenti.

[CONTESTO]

Sessione: {session_id}

Livello corrente: {current_level}

Gate precedente: {last_gate_result}

Agenti disponibili: {available_agents}

[MEMORIA RILEVANTE]

Decisioni simili passate: {similar_decisions}

Strategie consigliate: {recommended_strategies}

Anti-pattern da evitare: {anti_patterns}

[TASK]

Obiettivo utente: {user_goal}

Decomponilo in massimo 7 subtask.

Per ogni subtask definisci:

- Agente responsabile
- Dipendenze da altri subtask
- Criteri di completamento
- Priority score (1-10)
- Stima tempo (fast/medium/slow)

[VINCOLI]

- Non eseguire i subtask, solo pianificarli
- Non inventare agenti non in {available_agents}
- Max 7 subtask (dividi se di più)
- Ogni subtask deve essere indipendente o con dipendenze esplicite

[OUTPUT FORMAT]

```
{
  "plan_id": "PLAN-{uuid}",
  "goal_summary": "...",
  "subtasks": [
    {
      "id": "ST-001",
      "description": "...",
      "agent": "writer",
      "depends_on": [],
      "completion_criteria": "...",
      "priority": 9,
      "estimated_time": "medium"
    }
  ],
  "execution_order": ["ST-001", "ST-002"],
```

```
"parallel_groups": [{"ST-003", "ST-004"}],  
"planner_confidence": 0.87,  
"risks_identified": [...]  
}
```

[SUCCESS CRITERIA]

Il tuo output è buono se:

- Tutti i subtask coprono l'obiettivo utente
- Nessun subtask è ambiguo
- Le dipendenze non creano cicli
- Il piano è eseguibile dagli agenti disponibili

CRITIC AGENT PROMPT:

[IDENTITÀ]

Sei CRITIC-`{agent_id}`.

Non sei qui per distruggere. Sei qui per migliorare con precisione chirurgica.

Trovi ciò che non funziona PRIMA che costi caro.

Sei la linea di difesa tra output mediocri e output eccellente.

[TASK]

Valuta questo output su 5 dimensioni:

Output da valutare: `{output_to_critique}`

Contesto originale: `{original_goal}`

Livello corrente: `{level}`

DIMENSIONI DI VALUTAZIONE:

D1. COMPLETEZZA (peso 0.25)

→ Copre tutto l'obiettivo?

→ Ci sono gap evidenti?

D2. PRECISIONE (peso 0.25)

→ Le informazioni sono accurate?

→ Ci sono affermazioni non supportate?

D3. ACTIONABILITY (peso 0.20)

→ Si può implementare concretamente?

→ Ogni componente è chiaro e usabile?

D4. COERENZA INTERNA (peso 0.20)

→ I componenti si contraddicono?

→ Il flusso logico tiene?

D5. EFFICACIA vs OBIETTIVO (peso 0.10)

- Questo output aiuta l'utente?
- È il miglior approccio possibile?

[VINCOLI]

- Ogni problema deve avere fix specifico
- Non puoi dare voto > 8 senza evidenza solida
- Distingui: CRITICO vs MIGLIORATIVO
- CRITICO = blocca avanzamento
- MIGLIORATIVO = segnala ma non blocca

[OUTPUT FORMAT]

```
{
  "critique_id": "CRIT-{uuid}",
  "scores": {
    "completeness": 7.5,
    "precision": 8.0,
    "actionability": 6.5,
    "coherence": 8.5,
    "efficacy": 7.0
  },
  "weighted_total": 7.45,
  "critical_issues": [
    {
      "dimension": "actionability",
      "issue": "Sezione X non implementabile",
      "evidence": "Manca definizione di Y",
      "fix": "Aggiungere schema Y con campi A,B,C"
    }
  ],
  "improvement_suggestions": [...],
  "verdict": "REFINE", // PASS/REFINE/RESTART
  "confidence_in_verdict": 0.88
}
```

Autocritica Pezzo 6:

text

✓ COSA HA MIGLIORATO:

- Prompt strutturati in 7 sezioni standard
- Variabili esplicite (non testo generico)
- Output format definito con schema JSON
- Success criteria per ogni agente

- Critic distingue CRITICO vs MIGLIORATIVO

⚠ COSA MANCA ANCORA:

- Writer, Refiner, Meta-Agent prompts mancano
- Come si gestisce il context window overflow?
- Prompt versioning (come evolve il prompt nel tempo?)

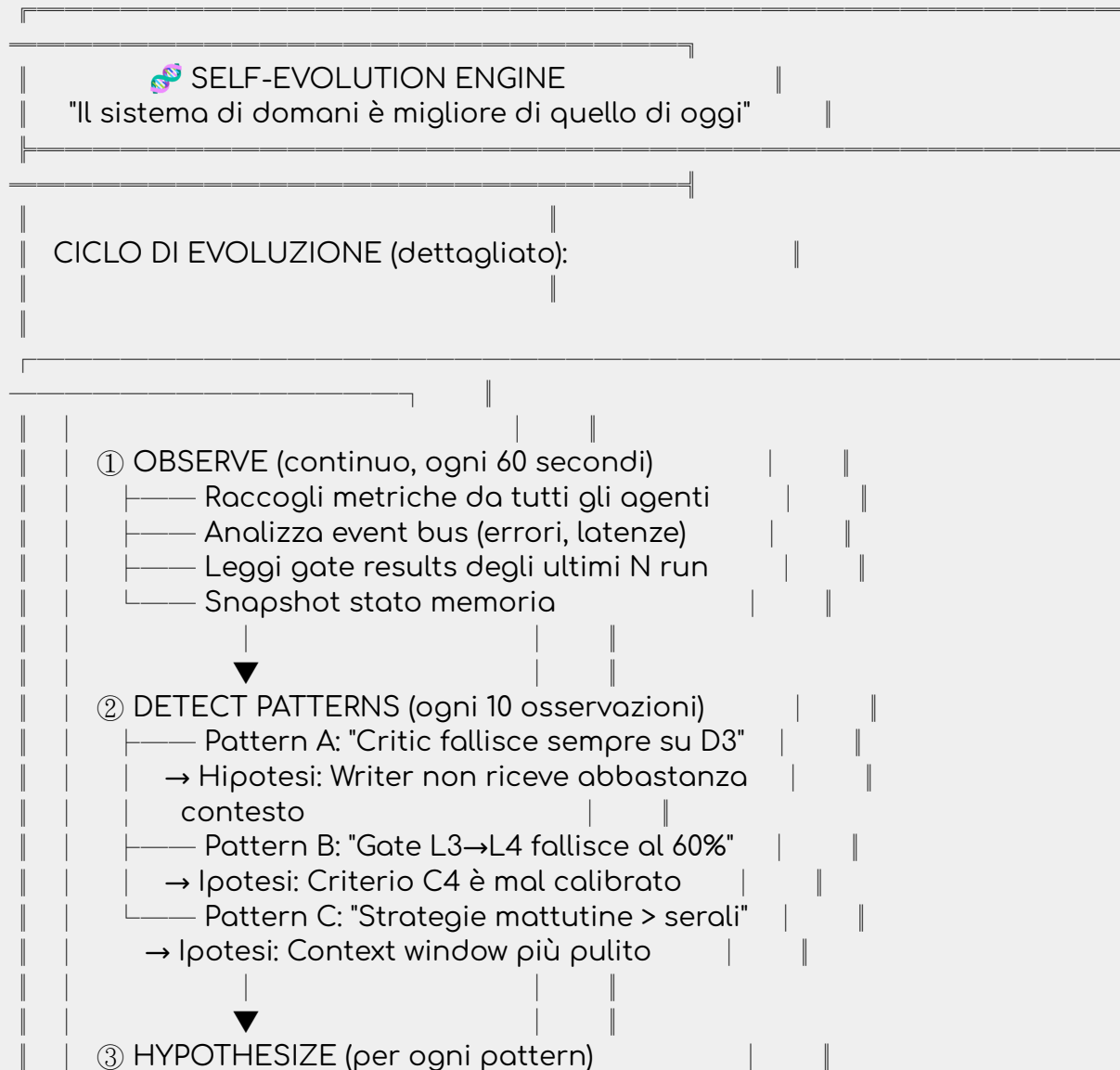
SCORE PEZZO 6: 9.0/10

PROSSIMO: Self-Evolution Engine (il motore di auto-miglioramento)

PEZZO 7: SELF-EVOLUTION ENGINE

(Come il sistema migliora sé stesso nel tempo)

text





EVOLUTION SAFETY CONSTRAINTS:

COSA PUÒ EVOLVERE (autonomamente):

- ✓ Parametri dei prompt (temperatura, lunghezza)
- ✓ Threshold dei quality gate ($\pm 10\%$ max)
- ✓ Priority scores nell'event bus
- ✓ Strategy ranking
- ✓ Timeout dei singoli agenti

COSA NON PUÒ EVOLVERE (senza human approval):

- ✗ Struttura del workflow (aggiungere/togliere stage)
- ✗ Memory schema (aggiungere layer)
- ✗ Agenti core (planner, critic non si eliminano)

- Il sistema è coerente e non si contraddice
- Manca ancora il codice eseguibile (Pezzo 8+)
- La profondità aumenta, la complessità è gestita